



DPUBirdsEye

DPUBirdsEye shows a schematic illustration of the current driving simulation from a bird's-eye perspective. Information concerning the road network and the objects in the landscape, the test driver, the autonomous traffic, and the pedestrian simulation can be shown.

1. Inputs:

<i>SizeY</i>	Height (in meters) of the database section that is displayed in the schematic view. The width of the section adjusts itself automatically according to the aspect ratio of the window.
<i>X</i>	The X-coordinate of the test driver's position.
<i>Y</i>	The Y-coordinate of the test driver's position.
<i>yaw</i>	Yaw angle of the test driver.
<i>DrawGrid</i>	Should a coordinate grid be displayed in the background of the view? The spacing of the grid lines is 10 m.
<i><additional entries></i>	Used to select which data is displayed. These inputs will be explained in the following pages.

2. Parameters:

<i>ScreenshotName</i>	Indicates in which directory and under which file name the screenshots are saved. If no absolute path is entered, the system will search for the folder below the SILAB installation directory. Default value: "Data\BEVScreenshot".
<i>ScreenshotOverwrite</i>	If true is entered, only one screenshot file will be generated (using the specified ScreenshotName) and overwritten by each following screenshot.
<i>ScreenshotTransparency</i>	The default value false generates sequentially numbered files. If true is entered, all screenshots contain transparency information (i.e., the image file contains an alpha channel). The default value false generates a completely opaque screenshot.
<i>ScreenshotOptions</i>	Additional options depending on the image file format. These options are documented in the SILABImageTool help function. Normally, an additional option is not required.
<i>Screenshot</i>	When changing this Boolean variable from 0 to 1, a screenshot is generated.

3. Static parameters:

<i>WindowX,WindowY</i>	Left upper corner of the preview window (screen coordinates).
<i>WindowW,WindowH</i>	Width or height of the preview window (screen coordinates).
<i>RenderOffscreen</i>	Defines which method to use in order to render the graphic display of DPUBirdsEye. See section "Off-screen Rendering". The default value is false (no off-screen rendering).



4. Prerequisites:

DPUBirdsEye requires an OpenGL compatible graphics card (e.g., NVIDIA GeForce 4 or better) with sufficient performance. (The required performance depends on which displays are selected. Text-based displays such as DrawVisObjectName require a lot of processing power.)

Furthermore, the directory `\silab\lib\graphics\default\birdseyeviewer` must exist on this computer and must contain the files *FontCourier.glf*, *FontNormal.glf* and *FontNormalItalics.glf*.

5. Functional principle:

DPUBirdsEye displays in a window a section of the database in a schematic and bird's-eye perspective (the databases SCN and SCNX are both supported). The driver is shown in the center of this window.

In order to monitor the sequence of the simulation, run the DPUBirdsEye on one of the simulation computers. The computer must have an OpenGL-capable graphics card installed. However, the requirements that must be met by the graphics card are not as stringent as those that are mandatory for a complete simulation view (compare to DPUSGEView).

In order for the DPUBirdsEye to display all changes during the current simulation step, it is recommended to assign the highest Index value to it. Moreover, the current position and the yaw angle of the driver must be transmitted to the DPUBirdsEye. An example configuration follows:

```
DPUBirdsEye bev
{
    Computer = { OPERATOR };
    Index = 100; # after all others

    # Width, height and position of the window
    WindowW = 800;
    WindowH = 600;
    WindowX = 0;
    WindowY = 0;
    # Size of the visible section of the database
    SizeY = 100;
};
Connections =
{
    VDyn.X <-> bev.X,
    VDyn.Y <-> bev.Y,
    VDyn.yaw <-> bev.yaw
};
```

DPUBirdsEye is capable of displaying a variety of data schematically from all simulation areas. In order to avoid display clutter, all display options can be switched on and off individually. The following text will explain what can be displayed within each simulation module and provide the name of each option. All Draw parameters are Boolean variables that can be set to 1 (true) or 0 (false).



6. Off-screen rendering:

Two options are provided on how to render a DPUBirdsEye display: "onscreen" and "off-screen". The "onscreen" method is selected by default. The off-screen method can be used as an alternative; however, it has its advantages and disadvantages:

- The off-screen method requires a graphics card that is OpenGL 3.0 compatible. (Requirements similar to DPUSGEView apply.) The onscreen methods works fine with older graphics cards.
- When using the off-screen method, changing the size of the drawing window subsequently is not possible (this is irrelevant for the DPUBirdsEye since the size of the window is fixed).
- The off-screen method lets you generate the correct screenshots if the BirdsEye window is hidden by (WINDOWS) frames or extends beyond the edge of the visible desktop. In this case, the onscreen method creates screenshots in which parts are black.
- Thus, the off-screen method lets you generate screenshots that are larger than the WINDOWS desktop. These types of screenshots are well suited for the printout of a road network poster. The maximum size of the screenshot that can be produced by this method depends on the graphics card installed. The latest NVidia graphic cards can create images of up to 8 mega pixels (2,800 x 2,800 pixels).

1 SCNX DATABASE

For the SCNX database, information about the surrounding landscape and the road network can be displayed.

The following illustration shows this view (the blue numbers are for information only and are not generated by DPUBirdsEye):

Illustration of the elevation model

The various elevations of the landscape can be displayed as background shading.

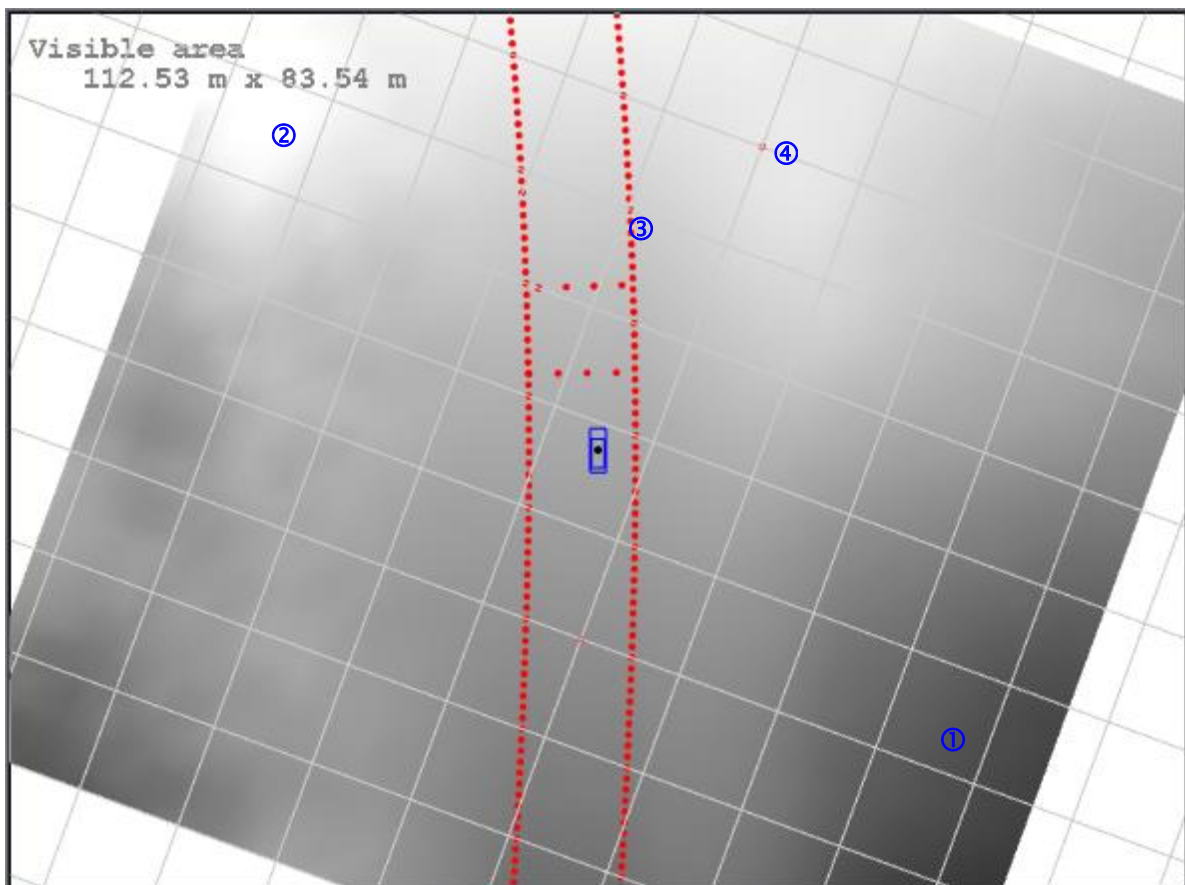


Figure 1: Illustration of the elevation model

DrawLandscape	Should the elevations of the landscape be displayed? Here, the low points of the landscape (1) are displayed in shades of dark gray, and the higher points (2) are shown as shades of brighter gray.
LandscapeRange	The side length of the square area (in meters) surrounding the driver, in which the heights of the landscape are displayed.
LandscapeStep	The distance between two points (in meters) for which the height information is queried. Between these points, the display uses linear interpolation.
Note: This parameter affects significantly the computation efforts of DPUBirdsEye! The smaller the value, the greater the computation effort.	
LandscapeColHeight	Height range (in meters) that will be displayed in the color sequence black -> white.
LandscapeColClamp	If the actual height range within the display area is larger than LandscapeColHeight, should the higher points be displayed in white (true) or should the range be enlarged automatically (false)?
DrawHFine	Displays information on how a road affects the elevation model. Points (3) identify the limit of the area affected by the road.
DrawHCoarse	Displays information concerning the rough elevation model (4).



Display of road sections

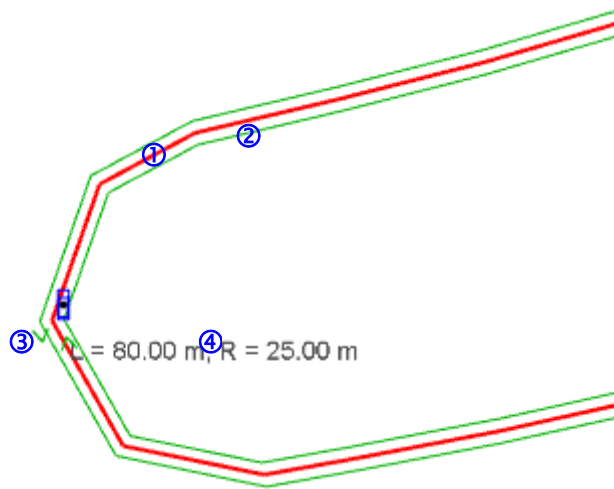


Figure 2: Display of course road segments

DrawRefCurve

Draws the reference curve (1) of the course segments. The reference curve is located in the center of the road. If the road runs at a level plane, a blue line represents the reference curve; running uphill, the line is red; running downhill, the line is green.

DrawCourseLanes

Draws all lanes (2) of the course segments. Lines are drawn in the centers of the lanes. Arrows (3) are used to identify the lane direction.

DrawRefCurveInfoText

Writes information pertaining to the length and the radius of the respective course segment (4).

CurveResolution

If true, the resolution of the lanes' curves will be increased.

DrawRoad

Draws the street as a grey polygon.

DrawRoadBorder

Draws a black line along the roadside.

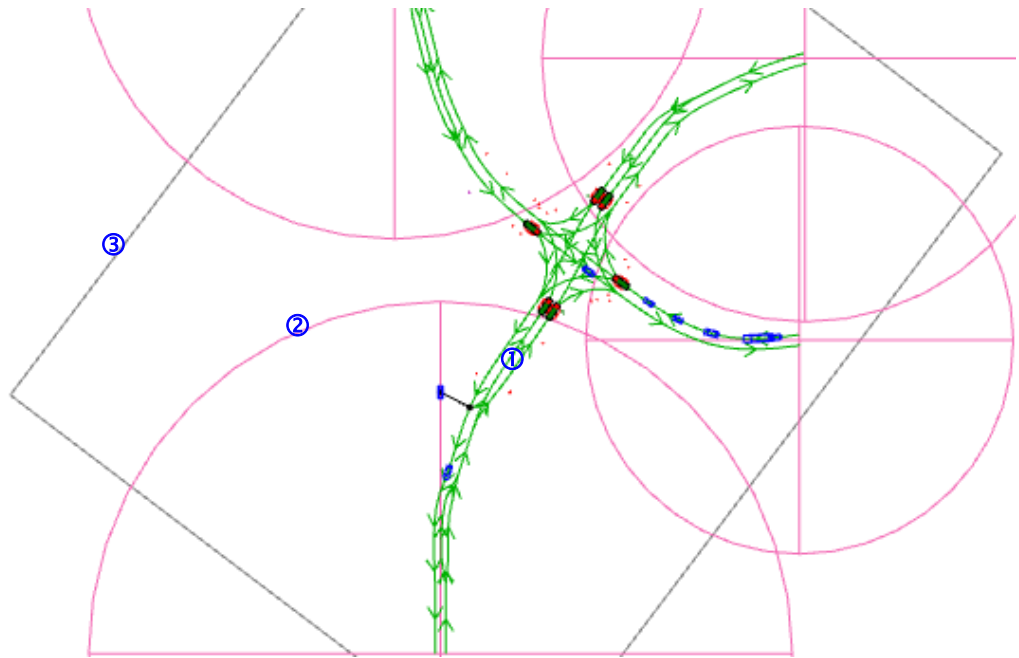
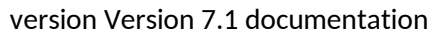


Figure 3: Illustration of Area2 road segments

DrawArea

DrawAreaLanes

DrawArea2Lanes

DrawArea2RVis

Draws rectangles (3) that enclose the Area2 element.

Draws the lanes (1) in Area road segments (SCN).

Draws the lanes (1) in Area2 road segments (SCNX).

Draws the visibility radii (2) for the ports of the Area2 elements.

Display of objects



Figure 4: Display of objects

DrawVisObjects

DrawLGVisObjects

DrawVisObjectName

Draws objects (1) that were placed manually along the course by the user (light blue).

Draws objects that were placed by the landscape generator (2) (dark blue).

Writes the names of (3) all objects.



Display of road events (hedgehogs)

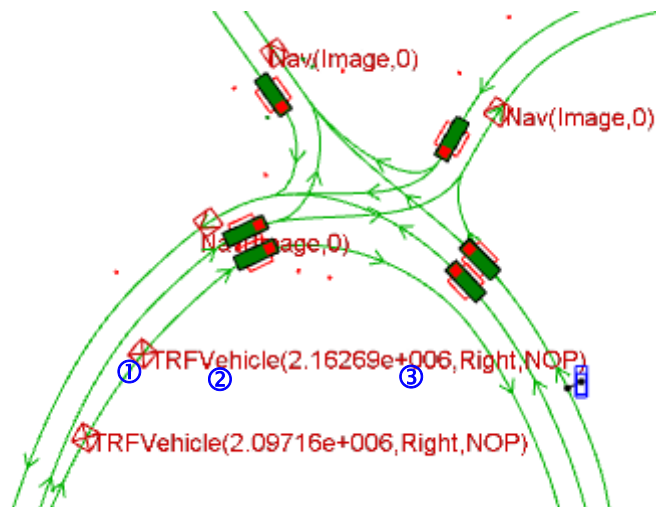


Figure 5: Display of road events (hedgehogs)

DrawHedgehogs

Draws a box (1) for each road event, which shows the position of the road event and which driving direction(s) it is valid for.

DrawHedgehogText

Provides detailed information about the road event.

DrawHedgehogFamily

Provides the family name (2) of the road event.

DrawHedgehogParams

Provides the parameters (3) of the road event.

Display of vehicle information

Information about the position and the direction of the simulation vehicle can be shown in the upper left corner of the BirdsEye display:

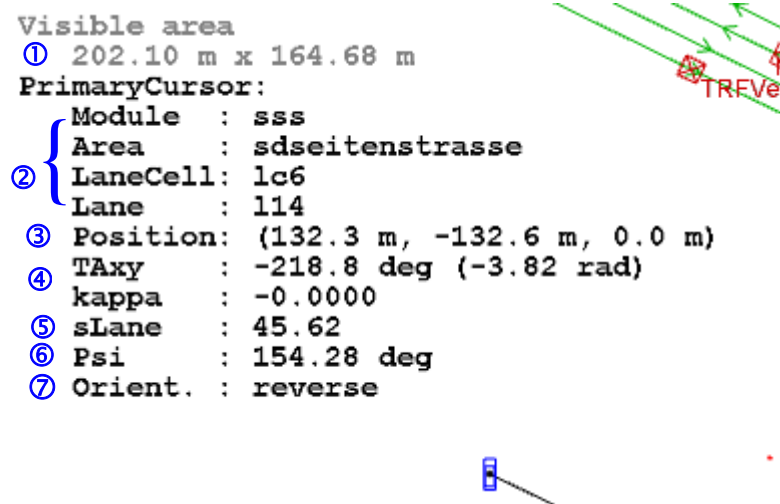


Figure 6: Display of vehicle information



The size of the visible area (1) is always shown.

<i>DrawPCCurrentNodesText</i>	Provides information in which module, road segment, and lane (2) the simulation vehicle is currently located.
<i>DrawPCPositionText</i>	Indicates the position (3) of the simulation vehicle in global coordinates.
<i>DrawPCTACurvText</i>	Indicates the tangent angle (4) of the road section in which the vehicle is located.
<i>DrawPCSText</i>	Indicates the distance along the road (5).
<i>DrawPCHeadingText</i>	Currently not used.
<i>DrawPCPsiText</i>	Indicates the yaw angle of the vehicle (6) relative to the tangent angle of the road.
<i>DrawPCOrientationText</i>	Indicates (7) whether the vehicle is moving in the direction of the lane or against the direction of its lane.

2 TRFX TRAFFIC

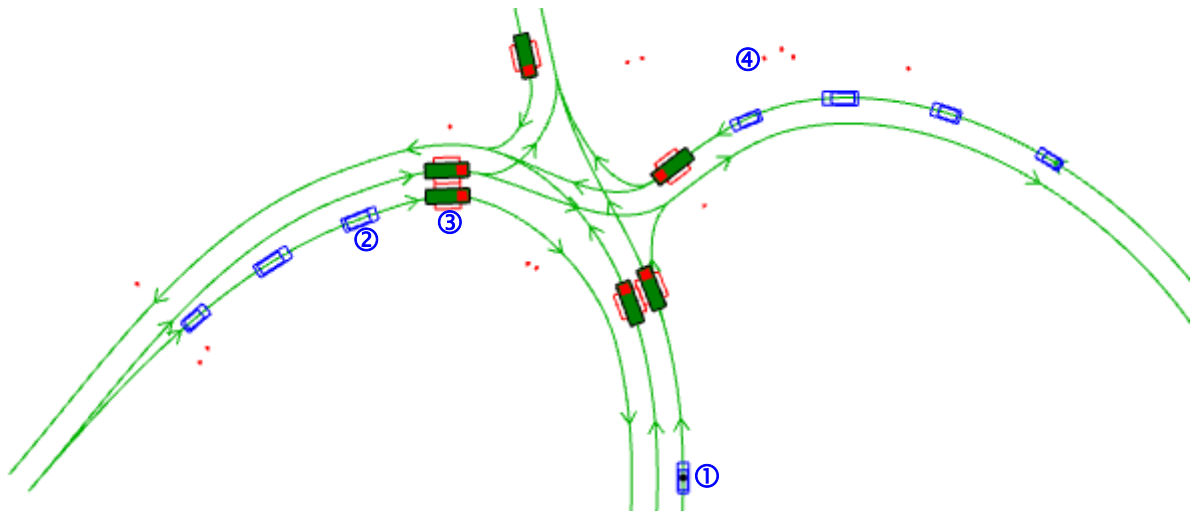


Figure 7: Default TRFX visualization

The vehicles of the traffic simulation (1)(2) are shown in blue. A black dot identifies the simulation vehicle (1).

Traffic lights (3), including their current signaling condition are shown schematically. Other obstacles that must be considered in the traffic situation (e.g. pedestrians) are shown as a red rectangle (4).

The following parameters control this basic traffic simulation display:

<i>DrawTRFSimCar</i>	The EGO vehicle is drawn as a blue box.
<i>DrawTRFRoadUser</i>	Third party vehicles are drawn as a blue box.
<i>DrawTRFObstacle</i>	Scan boxes (5) for miscellaneous obstacles and their respective bounding boxes are drawn.
<i>DrawTRFLights</i>	Traffic lights (3) are drawn.

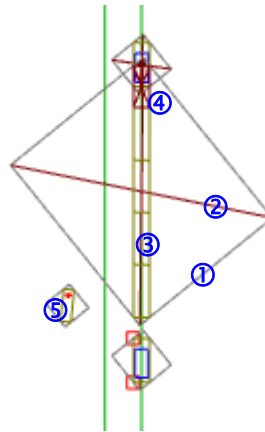


Figure 8: Additional traffic visualization

DrawTRFBB

Bounding box (1) for all scan boxes (look-ahead boxes used by the traffic simulation) are drawn.

DrawTRFBBCollision

Intersecting bounding boxes (2) are crossed out in red to alert of an imminent collision.

DrawTRFScanBox

Scan boxes (3) for all vehicles are drawn.

DrawTRFScanBoxCollision

Intersecting scan boxes (4) are crossed out in red to alert of an imminent collision.

DrawTRFID

Red letters next to each vehicle identify its TRF ID and User ID (if assigned).

DrawTRFSimCarBox

Scan boxes for the EGO vehicle are drawn.

DrawTRFCoDrivingBox

Scan boxes representing additional human drivers are drawn.

3 PEDESTRIAN SIMULATION MOV

The following information can be displayed for the pedestrian simulation MOV:

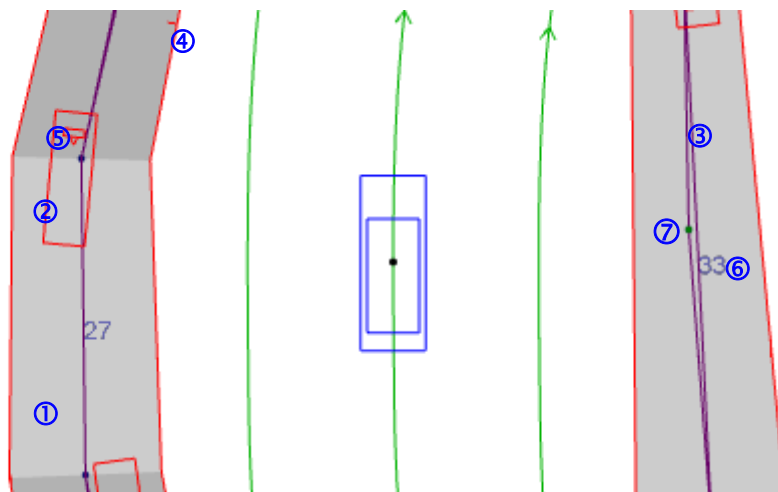




Figure 9 Pedestrian visualization

<i>DrawMovArea</i>	Draws pedestrian areas (1). Pedestrians normally do not leave these areas.
<i>DrawMovBoxes</i>	Draws scan boxes (2) for each pedestrian.
<i>DrawMovGraph</i>	Draws the path planning graph (3). The pedestrians walk near those lines.
<i>DrawMovObstacles</i>	Draws the outline of an obstacle (4) (walls and obstacles).
<i>DrawMovPersons</i>	Draws the pedestrian (5). Walking pedestrians are drawn in red, while pedestrians waiting at a traffic light are drawn in green, and while attempting an evasive action, they are drawn in violet.
<i>DrawMovPolyID</i>	Provides the numerical identification (6) for each polygon of the area safe to walk on.
<i>DrawMovStartPos</i>	Draws the start and target points (7) (in green).

4 OBJECT DATABASE ODB

The queries requested from the object database and their results can also be visualized.

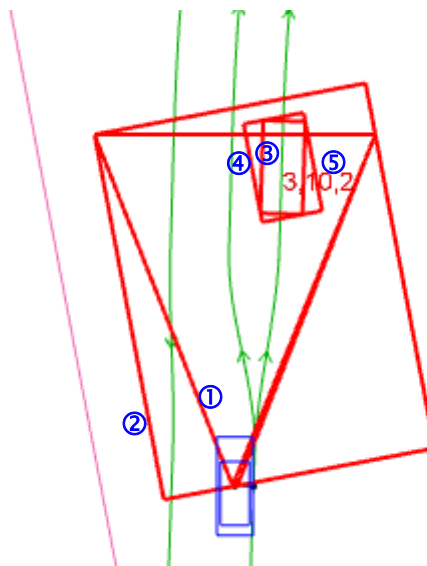


Figure 10: Visualization of object database queries

The following parameters can be used for this purpose:

<i>DrawODBQuery</i>	Draws the contours of all ODB query polygons (1) in form of a line. Queries that do not intersect with an ODB object are displayed in green; in case of an intersection, the line appears in red.
---------------------	---



<code>DrawODBQueryAABB</code>	Draws an axis-aligned bounding box (2) that surrounds the ODB query polygon.
<code>DrawODBQueryResults</code>	Draws the contours of all objects (3) that intersect an ODB query.
<code>DrawODBQueryResultsAABB</code>	Draws an axis-aligned bounding box (4) that surrounds all objects that intersect with the ODB query.
<code>DrawODBQueryResultsType</code>	Provides the ODB object type (5) (shown as three digits) for all objects that intersect with an ODB query. A list of the object types can be found in the "Reference Object database ODB" documentation.

5 CONFIGURATION SECTION SILAB BIRDSEYE

In addition to the DPUBirdsEye configuration options – which can be dynamically modified via DPU inputs – numerous static parameters are available that can be entered in the separate *SILAB BirdsEye* configuration section.

The configuration section must be placed on the top level of the test configuration. It contains several subsections that are named *BirdsEye <module_name>* for the individual modules of the BirdsEye visualization.

For example, it is possible to modify the colors of the individual graphical elements in each section.

The following is an example of a SILAB BirdsEye configuration section:

```
SILAB BirdsEye
{
    BirdsEye TRF
    {
        ColRoadUser = {0,0,0};
        #          TRFID, R, G, B,
        MarkRoadUser = {0,  0.5, 0.5, 1,
                        1,  1, 0.5, 0.5,
                        2,  1, 1, 0.5,
                        3,  0.5, 1, 0.5,
                        4,  0.5, 1, 1};
        #          UserID, R, G, B,
        MarkUserID = {2312, 0, 1, 0};
    };

    BirdsEye SCNX
    {
        ColPC = {0,0,1};
    };
};
```

The value of a color will be defined as follows:

```
ColorName = { <Red>, <Green>, <Blue> };
```



Here, <Red>, <Green> and <Blue> are the individual color components, with possible values ranging from 0 to 1.

5.1 Parameter of the BirdsEye TRF section:

<i>ColRoadUser</i>	= {R,G,B}: The colors used for drawing vehicles. Default: blue.
<i>MarkRoadUser</i>	= {ID1, R1, G1, B1, ...}: A list of color assignments. The colors {Rn, Gn, Bn} are assigned to an internal traffic ID, IDn. The associated vehicle will be filled with this color. Any number of comma-separated color assignments can be specified in this list.
<i>MarkUserID</i>	= {UID1, R1, G1, B1, ...}: A list of color assignments. The color {Rn, Gn, Bn} will be assigned to all vehicles with a UIDn user ID. The associated vehicle will be filled with this color. Any number of comma-separated color assignments can be specified in this list.
<i>MarkOpacity</i>	Opacity of the vehicle markers, in the value range 0 (transparent) to 1 (opaque), default value 0.5.

5.2 Parameter of the BirdsEye SCNX section:

<i>ColPC</i>	= {R,G,B}: Color of the primary cursor. Default: black.
<i>ColBottom</i>	= {R,G,B}: Color that identifies valleys in the landscape (if DrawLandscape = 1). Default: black.
<i>ColTop</i>	= {R,G,B}: Color that identifies hills in the landscape (if DrawLandscape = 1). Default: white.
<i>FillArea</i>	= false (default)/true: Indicates whether the area occupied by an Area2 road segment should be filled in (otherwise, a border will be drawn).
<i>ColArea</i>	= {R,G,B}: Color that identifies Area2 road segments. Default: grey
<i>ColRoad</i>	= {R,G,B}: Color that identifies a road. Default: grey
<i>ColBigRoad</i>	= {R,G,B}: Color that identifies a multi-lane road, with several lanes travelling in the same direction. Default: grey
<i>FillVisObjects</i>	= false (default)/true: Indicates whether objects of the landscape should be filled in (otherwise, a border will be drawn)
<i>ColVisObjects</i>	= {R,G,B}: Color identifying individually placed objects of the landscape. Default: blue.
<i>ColLGVisObjects</i>	= {R,G,B}: Color identifying objects that are automatically placed by the landscape generator. Default: light blue.
<i>ColTreeObjects</i>	= {R,G,B}: Color identifying trees. Default: blue.
<i>ColFieldObjects</i>	= {R,G,B}: Color identifying fields. Default: blue.
<i>ColHouseObjects</i>	= {R,G,B}: Color identifying houses. Default: blue.

6 REALISTIC MAP DISPLAY

When using the appropriate DPUBirdsEye configuration, the display can be adjusted to visualize the road network as a realistic 2D map.

The display is similar to conventional maps with a high degree of detail, such as city maps, Internet map service providers (e.g., Google Maps) or navigation systems.



First, enter the following lines into the DPU configuration of your project in order to include DPUBirdsEye and to set the DPU parameters correctly (see Chapters 1 and 2):

```
include system/DPUBirdsEye.inc

bev.DrawArea2Lanes = false;
bev.DrawArea2RVis = false;
bev.DrawCourseLanes = false;
bev.DrawRefCurve = false;
bev.DrawVisObjects = true;
bev.DrawLGVisObjects = true;
bev.DrawTRFSimCar = false;
bev.DrawTRFRoadUser = false;
bev.DrawTRFID = false;
bev.DrawTRFLights = false;
bev.DrawPOI = false;
bev.DrawRoad = true;
bev.DrawRoadBorder = true;
```

Now, in order to proceed with additional color adjustments, enter the following configuration section in the top level:

```
SILAB BirdsEye
{
    ColBackground = {0.85, 0.85, 0.85, 1};
    VisAreaText = false;

    BirdsEye SCNX
    {
        PC_Size = 8;

        ColRoad = {1, 1, 1};
        ColBigRoad = {1, 0.75, 0.27};
        ColRoadBorder = {0.65, 0.65, 0.65};

        ColArea = {0.8, 0.75, 0.7, 0.2};
        FillArea = true;

        FillVisObjects = true;

        ColTreeObjects = { 0.7, 0.85, 0.65 };
        ColHouseObjects = { 0.8, 0.75, 0.7 };
        ColFieldObjects = { 0.85, 0.85, 0.7 };

        ColVisObjects = {0.8, 0.8, 0.8};
```



SILAB DRIVING SIMULATION

version Version 7.1 documentation

```
CollGVisObjects = {0.8, 0.8, 0.8};  
};  
};
```

An example of the resulting display can be seen in the following screenshot:

